
aCelery

User's Guide

Build and run your own JavaScript apps on your phone.

Version 1.0.5 · GPLv3 · www.acelery.com

aCelery is a small development environment that lives on your device. You write an app in the built-in editor, press Run, and it opens — no computer, no build step, no account, no app store. An app is a folder of plain files: a manifest, an entry module, whatever CSS and pictures you want. It talks to SQLite, the device's filesystem and the network through a small set of capabilities that aCelery provides.

This guide has two halves.

- **Part 1 — Using aCelery** describes the app itself: the five screens, running apps, sharing them with other devices on your network, and connecting an AI assistant.
- **Part 2 — Writing aCelery apps** is the reference for everything an app can import and call.

Licence: GNU General Public License v3.0.

Contents

Part 1 — Using aCelery

1. What aCelery is
2. Getting around
3. Home
4. Apps
5. Code
6. Data
7. Settings
8. Running an app
9. Sharing on your network
10. Connecting an AI assistant
11. Where your work is stored

Part 2 — Writing aCelery apps

12. An app, end to end
13. The anatomy of an app
14. Rendering: Preact and htm
15. `acelery/ui.js` — the widget layer
16. `TableMaint` — a CRUD screen from a declaration
17. `acelery/sql.js` — SQLite
18. `acelery/file.js` — files
19. `acelery/picker.js` — files and photos the user picks
20. `acelery/http.js` — the network
21. `acelery/export.js` — sharing files and leaving the app
22. `acelery/chart.js` — charts
23. `acelery/datatable.js` — sortable, searchable tables
24. Styling and theming your app
25. Debugging
26. Limits, rules and gotchas

Part 1 — Using aCelery

1. What aCelery is

aCelery runs a tiny web server inside itself, on port `8123`, and shows you pages from it. The system screens you use — Home, Apps, Code, Data, Settings — are built the same way your own apps are, out of the same components, with no privileged access to anything.

That one design decision is why aCelery can do something unusual: because everything already travels over HTTP, you can open aCelery, and your own apps, from a browser on another computer on the same Wi-Fi. It is the same server answering, not a second implementation. See [Sharing on your network](#).

Two things follow from it that are worth knowing up front:

- Your apps run in a WebView — the device's own browser engine. Anything the browser can do, your app can do.
- Nothing is bundled or minified. The file you write is the file that runs, so an error message points at your line.

2. Getting around

aCelery has five destinations. On a phone they are along the bottom; on a tablet or a desktop browser they are a rail down the left.

Destination	What it is for
Home	Pick up where you left off, or start something new
Apps	Everything you can run
Code	Projects, files and the editor
Data	Databases, tables and SQL
Settings	Theme, network access, about

Every screen has an address. `#/code/Example/main.js` is the editor open on that file, `#/data/notes.db/table/note` is that table. The device's Back button steps back through them the way a browser does, and a reload lands you where you were — including after you run an app and come back.

On a wide screen (992 px and up) Code shows the file list and the editor side by side; on a phone they are separate screens.

3. Home

Home opens on what you were last doing.

- **Continue** shows the project and the database you touched most recently, with **Open** and **Run** on the project. If you deleted one since, the card quietly disappears rather than leading nowhere.
- **Create** has **New app** and **New database**, which take you to Code or Data with the dialog already open.

- **On this device** counts your apps and databases, and each count is a link.

The first time you open aCelery there is nothing to continue, so Home offers to run the **Example** app or create your first one. The Example app is the reference implementation — every feature in Part 2 of this guide is demonstrated in it, and its source is one readable file you can open in Code.

4. Apps

Everything runnable, as cards. Tapping a card runs the app.

Each card's menu offers:

- **Edit in Code** — opens the project in the editor.
- **Add to home screen** — puts an icon for this app on the device's home screen, so it opens straight into your app. *Android only; iOS gives apps no way to do this.*
- **Export** — zips the project and hands it to the share sheet, so you can mail it to yourself, save it to Files, or send it to another phone.
- **Delete** — removes the app folder and everything in it. It does not delete databases the app used; those live in Data.

Once you have more than six apps a search box appears, matching on name and description.

5. Code

Code opens on your projects. **New project** asks for a name (letters, numbers and underscore, 16 characters at most) and an optional one-line description, then creates the folder with a manifest and a `main.js` that already runs, and drops you into the editor.

Import project (the icon in the top bar) reads a project zip back — one exported from here, or from another device. *Only inside the aCelery app: a browser on the network has no access to the device's file picker.*

The project screen

A project is its list of files. Tapping one opens it in the editor.

- **New file** creates a `.js` or a `.css` in the project.
- **Run** launches the app. From Code this is a *debug* run, which adds the Error log to the running app's menu.
- The project menu also has **Export project**, **Delete (this file)** and **Delete project**.

Pictures preview instead of opening. Files that are not text — zips, fonts, audio, databases — say so rather than opening as garbage in the editor.

The editor

CodeMirror 6, chosen because it is the only major editor with practical touch support. It gives you line numbers, syntax highlighting, bracket matching and closing, code folding, undo history, search, autocompletion and an active-line highlight.

Highlighting is by extension: `.js`, `.mjs`, `.json`, `.css`, `.html`, `.htm`, `.xml` and `.svg`. Anything else opens with HTML highlighting.

A dot beside the file name means unsaved changes. **Save** is the disk icon, but you rarely need it: running the app saves first, and so does leaving aCelery or backing out of the editor with the device's Back button. Navigating away inside aCelery with unsaved work asks whether to **Save** or **Discard** — dismissing the question leaves you where you are.

6. Data

Data opens on your SQLite databases. **New database** creates one; the name takes letters, numbers, dash and underscore, and the file is saved with a `.db` extension.

Opening a database gives you two tabs.

Tables

Every table in the database. Opening one gives you the same browser your own apps get from `TableMaint`: a list, a record view, editing, search and paging, built from the table's real columns. You can choose which columns the list shows, jump to **Structure** to see the column definitions SQLite holds, or **Drop table**.

This is a data browser and editor. It is meant for looking at what your app stored, fixing a value by hand, and clearing test rows.

SQL

A scratchpad. Type a statement and press **Run**, or Ctrl/Cmd + Enter.

- `SELECT`, `PRAGMA`, `WITH` and `EXPLAIN` show their rows, with the row count and how long it took. The first 200 rows render, with **Show all** for the rest.
- `INSERT` reports the new row id.
- Anything else reports the number of rows changed.

History keeps the last ten statements of the session, so you can bring one back rather than retype it.

The database menu has **Delete database**, which removes the file.

Create your tables from your app, with `create table if not exists` when it starts. Use the SQL tab to look at and repair data, not to set up the schema your app depends on — otherwise the app will not work on a device where you have not run those statements by hand.

7. Settings

Changes apply as you make them; there is nothing to save.

Appearance

- **Mode** — System, Light or Dark. It applies to the two themes that can render either way: *aCelery* and *Default*. A Bootswatch theme is one or the other, and with one of those selected the Mode control tells you so.
- **Theme** — 18 of them: *aCelery*, *Default*, *Cerulean*, *Cosmo*, *Cyborg*, *Darkly*, *Flatly*, *Journal*, *Lumen*, *Paper*, *Readable*, *Sandstone*, *Simplex*, *Slate*, *Spacelab*, *Superhero*, *United* and *Yeti*. *Cyborg*, *Darkly*, *Slate* and *Superhero* are dark. The theme applies to your own apps as well as to aCelery's screens.
- **Editor colours** — the code editor's own scheme. Left as *Light or dark, to match the app*, it follows the Mode; choose a palette and it stays that palette whatever the Mode does.

This device

(Not shown in a browser on the network — these are the device's own settings.)

- **Network access** — opens the sharing sheet. See the next section.
- **Keep screen on** — stops the device sleeping while aCelery is open. Useful while you are reading code on screen, and necessary on iPhone and iPad if you want the server to stay reachable.

About

The version and licence, the address aCelery is serving on, and a link to www.acelery.com.

8. Running an app

Running an app opens it full-screen on top of aCelery, with its own title bar. The menu there has:

- **Reload** — restarts the app with the files as they are now. After an edit, this is the fastest loop.
- **Error log** — everything the app logged and every error it did not catch. Offered on a *debug* run, which is what **Run** from Code does.
- **Add to home screen** (*Android*).
- **Close** — back to aCelery. The device's Back button does the same.

If the app fails to start — a syntax error, a missing entry module, a throw inside `main` — aCelery shows the failure and its stack in place of the app, naming the file and the line. Nothing is bundled, so that line is your line.

9. Sharing on your network

Settings → **Network access**, or the same sheet from anywhere else that offers it.

By default aCelery listens only to itself. Turning on **Share on this network** makes it reachable from other devices on the same Wi-Fi, at the address the sheet then shows — something like `http://192.168.1.24:8123`. Open that in a browser on your computer and you get aCelery: the same screens, the same editor, your apps.

Pairing

A device that has never connected does not get in. It sees a page telling it to ask, showing a short code; a prompt appears on the phone with that device's address and the same code, and you approve or deny it. The codes are how you tell two simultaneous requests apart — approve the one showing the code your browser is showing.

Approving issues that device a key, so it does not have to ask again. The address alone is never the credential: addresses get reassigned, and anything on the network can claim one.

Approved devices lists everything currently holding a key, when it last called, and gives each one an × to revoke it.

Revoke all clears the lot.

What to keep in mind

- Approving a device gives it your databases and files. Approve devices you own, on networks you trust.
- Traffic is plain HTTP on your local network — no tunnel, no encryption. This is a LAN feature, not a way to publish an app on the internet.
- **On Android**, sharing keeps working with the screen off: while the switch is on, aCelery runs a foreground service and shows a notification with a **Stop sharing** button.

- **On iPhone and iPad**, iOS suspends the app within seconds of it leaving the screen, and offers nothing equivalent. Sharing works while aCelery is open on screen; turn on **Keep screen on** for longer sessions.

10. Connecting an AI assistant

aCelery can be driven by an AI assistant running on your computer — Claude Code, Claude Desktop, or anything else that speaks MCP. The assistant writes files into your projects, runs them on the phone, reads the console and the rendered page back, and fixes what it finds.

Settings → **Network access** → **Connect an assistant**. Name the key after where it will be used ("Claude on my laptop"), and aCelery shows you, once:

- the **address** to connect to, `http://<phone>:8123/mcp` ;
- the **key**;
- a ready-made **Claude Code** command line to paste into a terminal;
- a ready-made **Claude Desktop** configuration block.

Turn on **Share on this network** first, or the computer cannot reach the phone at all.

The assistant gets a set of tools over that connection: list, read, create and delete apps and their files; list databases, query them and run statements against them; and run an app on the device, read its console, inspect what it rendered, evaluate JavaScript in it, take a screenshot (*Android*) and close it. It also gets a written guide to the aCelery API — the same material as Part 2 of this document.

Keep in mind

- **The key is shown only once.** If you lose it, revoke it and make another.
- Anyone holding the key can read and change your apps and databases, and it travels unencrypted on your Wi-Fi. Use it on a network you trust, and revoke it in **Approved devices** when you are done.
- Being paired as a *browser* does not make a device an assistant, and being an assistant does not depend on the browser pairing. They are separate keys.
- On iOS the connection drops whenever aCelery leaves the screen.

11. Where your work is stored

Everything lives under one `aCelery` folder inside the app's own private storage — no permissions needed, and nothing else on the device can read it.

```
aCelery/  
  www/user/<App>/  your apps: one folder each  
  db/              your SQLite databases  
  files/           what your apps write with acelery/file.js  
  log/             error logs  
  cache/           scratch space (exported zips)
```

Uninstalling aCelery deletes all of it. **Export** your projects if they matter; that is the backup.

An app update never overwrites those four directories, so your projects, databases and files survive it.

Part 2 — Writing aCelery apps

12. An app, end to end

Here is a complete, working app. Create a project called `Notes`, put this in `main.js`, and press Run.

```
import { openDB } from "acelery/sql.js";
import {
  html, render, useState, useEffect,
  Panel, Form, Input, Button, ListGroup, notEmpty,
} from "acelery/ui.js";

function Notes({ db }) {
  const [notes, setNotes] = useState([]);

  const load = async () =>
    setNotes(await db.select("select rowid, body from note order by rowid desc"));

  useEffect(() => { load(); }, []);

  async function add(values) {
    await db.insert("insert into note (body) values (?)", [values.body]);
    await load();
  }

  return html`
    <${Panel} title="Notes">
      <${Form} initial=${${{ body: "" }} onSubmit=${add}}>
        <${Input} label="Note" name="body" validate=${[notEmpty()]} />
        <${Button} type="submit" variant="primary">Add</>
      </>
      <${ListGroup} variant="flush">
        ${notes.map((n) => html`<${ListGroup.Item} key=${n.rowid}>${n.body}</>`)}
      </>
    </>`;
}

export default async function main() {
  const db = await openDB("notes.db");
  await db.exec("create table if not exists note (body text)");
  render(html`<${Notes} db=${db} />`, document.body);
}
```

That is the whole shape of an aCelery app: two bare-name imports, a component tree, an exported `main`. Everything that follows is detail.

13. The anatomy of an app

```
www/user/<App>/
  acelery_app.json  the manifest
  main.js           the entry module (the manifest may name another file)
  *.css             optional; every .css file in the folder is loaded
  ...               anything else: more modules, pictures, data files
```

The manifest

```
{
  "name": "Water",
  "description": "Logs how much I drink",
  "entry": "main.js",
  "icon": "icon.png"
}
```

Key	Meaning
name	Shown on the card. Letters, digits and underscore, 16 at most
description	One line, shown under the name
entry	The module to run. <code>main.js</code> if absent
icon	A picture inside the project folder. Without one, a monogram is drawn

The `icon` path must stay inside the project — anything that climbs out of it, or names another origin, is ignored.

The entry module

aCelery imports the entry module and calls its **default export**:

```
export default function main() { ... }
```

`main` may be `async`, and aCelery waits for it. A named `main` export, or a global `main`, is accepted as a fallback, so exporting the wrong thing gives you your app rather than a blank page.

The page belongs to you: render into `document.body`. Other modules in the folder come in by relative path — `import { total } from "./sums.js";`.

Stylesheets

Every `.css` file in the project folder is loaded automatically, in the order the folder lists them. You do not link them.

Imports

Modules come from bare `acelery/` names, resolved by an import map:

Import	What it gives you
<code>acelery/ui.js</code>	rendering, hooks, components, forms, theming
<code>acelery/sql.js</code>	SQLite

Import	What it gives you
<code>acelery/file.js</code>	files under the aCelery folder
<code>acelery/picker.js</code>	picking files and photos; cropping and shrinking pictures
<code>acelery/http.js</code>	outbound HTTP through the device
<code>acelery/export.js</code>	share a file, leave the app
<code>acelery/chart.js</code>	charts — a separate import because Chart.js is large
<code>acelery/datatable.js</code>	sortable, searchable, paged tables — also a separate import
<code>acelery/editor.js</code>	the code editor, if your app wants one

There is nothing else to import. No npm, no CDN: the device may well be offline, and there is no install step to run. Everything in the table above is already on the device.

14. Rendering: Preact and htm

The UI is [Preact](#) — React's API in 4 KB — with [htm](#) templates instead of JSX. htm compiles its templates at runtime, which is precisely why an aCelery app needs no build step.

If you know JSX, you know this, with four differences:

```
html`
  <${Panel} title="Directory">
    <${Button} variant="primary" onClick=${save}>Save</>
    <div class="mt-3" ...${rest}>
      ${items.map((i) => html`<p key=${i.id}>${i.name}</p>`)}
    </div>
  </>`
```

- a component is interpolated: `<${Button}>`
- `</>` closes whatever component is open — you do not repeat its name
- attributes take expressions: `onClick=${save}`, `rows=${3}`, `disabled=${busy}`
- `class` and `className` both work on plain elements

State is hooks, imported from the same module: `useState`, `useEffect`, `useMemo`, `useRef`, `useCallback`, `useContext`, `useReducer`, `useLayoutEffect`. Also `render`, `createRef`, `Fragment` and `createContext`.

Change state to move between screens. Do not clear and rebuild the DOM by hand: the whole point of a component tree is that redrawing a list does not destroy the scroll position, the focus and the half-typed input inside it.

15. `acelery/ui.js` — the widget layer

One import gives you everything below.

Layout

Component	Notes
Container	Bootstrap's container
Row	A grid row; children wrap rather than overflow
Col span=\${6}	span is the width out of 12 on a tablet and up; on a phone every column is full width
Panel title="..." footer=\${...}	A card with a header. title and footer are optional
Card	Bootstrap's card, if you want to build the parts yourself

Forms

Form holds the values and validates them. onSubmit receives every field as one object, and fires only once every field passes.

```
<${Form} initial=${{ name: "", grp: "work", active: true }} onSubmit=${save}>
  <${Input} label="Name" name="name" validate=${[notEmpty()]} />
  <${Input} label="Email" name="email" type="email" validate=${[email()]} />
  <${Select} label="Group" name="grp" options=${[["work", "home"]]} />
  <${TextArea} label="Notes" name="notes" rows=${3} />
  <${CheckBox} label="Active" name="active" />
  <${Button} type="submit" variant="primary">Save</>
</>
```

Component	Props
Form	initial, onSubmit
Input	label, name, type, validate, help, placeholder, id, plus anything an <input> takes
TextArea	as Input, plus rows (default 4)
Select	as Input, plus options and placeholder
CheckBox	label, name
InputGroup	Bootstrap's input group, for a control with text or a button attached

Select options take bare strings, or { label, value } objects.

Each field owns its own <label for> / id pairing — you never write an id unless you want one. A field's error clears as soon as it is edited and is re-checked on submit.

Validators are notEmpty(), notZero(), email(), tel() and maxLength(n), each taking an optional message:

```
validate=${[notEmpty("Give it a name"), maxLength(40)]}
```

A validator of your own is just a function returning true when the value is good and a message when it is not:

```
const inThePast = (v) => new Date(v) <= new Date() ? true : "Not in the future";
```

`useForm()` reads the form from a component inside it: `{ values, errors, setValue, submitted }`.

Components from Bootstrap

Re-exported from `react-bootstrap` so you need one import. Their props are `react-bootstrap`'s.

`Alert` · `Badge` · `Button` · `ButtonGroup` · `Card` · `Container` · `Dropdown` · `DropdownButton` · `Image` · `InputGroup` · `ListGroup` · `Modal` · `Nav` · `NavDropdown` · `Navbar` · `Offcanvas` · `Pagination` · `Placeholder` · `ProgressBar` · `Spinner` · `Tab` · `Table` · `Tabs` · `Toast` · `ToastContainer`

aCelery's own

Export	What it does
<code>TableMaint</code>	A whole CRUD screen over one table — see the next section
<code>useTableOptions(db, table, column)</code>	Options for a <code>list</code> field, read from another table
<code>FileButton</code>	A button that opens the file or photo picker
<code>ImageCropper</code>	A dialog to drag and pinch a frame over a picture
<code>useDismiss(ref, onDismiss, active)</code>	Closes a panel when the user taps outside it, or presses <code>Escape</code>
<code>ThemeSelect</code>	A ready-made theme picker
<code>applyTheme</code> , <code>currentTheme</code> , <code>currentMode</code> , <code>themeHasModes</code> , <code>isDark</code> , <code>THEMES</code> , <code>MODES</code>	Theming, by hand

`FileButton` is the right way to offer a picker:

```
<${FileButton} accept="image/*" capture="environment"
  onFiles=${([photo]) => setPhoto(photo)}>Take a photo</>
```

Props: `onFiles` (called with a `File[]`; not called on cancel), `accept`, `multiple`, `capture` (`"environment"` for the back camera, `"user"` for the front), `variant`, `size`, `disabled`.

`ImageCropper` is open while its `image` prop is set:

```
<${ImageCropper} image=${picked} shape="round" aspect=${1} maxSide=${800}
  onDone=${(blob) => save(blob)} onCancel=${() => setPicked(null)} />
```

Props: `image` (a `File`, `Blob` or `URL`), `shape` (`"rect"` or `"round"`), `aspect`, `maxSide`, `type`, `quality`, `title`, `confirmLabel`, `onDone`, `onCancel`. A round frame still gives you a square picture; show it round with CSS.

16. `TableMaint` — a CRUD screen from a declaration

This is aCelery's distinguishing feature. Declare a table's fields and you get a working list, record view, editor, search and paging over it — the same screens the Data tab uses.

```
const FIELDS = [
  { type: "string", title: "Name", name: "mname", validate: [notEmpty()] },
  { type: "email", title: "Email", name: "email", validate: [email()] },
  { type: "list", title: "Group", name: "grp", options: ["family", "work"] },
];

<${TableMaint} db=${db} title="Directory" table="person" fields=${FIELDS} />
```

The table must already exist. Create it before you render — see [acelery/sql.js](#).

The views

- **List** — the rows, with New, First, Prev., Next, Last and Search.
- **Record** — one row, with Edit, Delete, Ok, and any special actions you added.
- **Edit / New** — the fields as inputs, with Save and Cancel.
- **Search** — tick the fields to search on, fill in a value or a range.

Paging walks row ids rather than counting offsets, so Next and Prev stay cheap on a table of any size. `pageSize` defaults to 20.

Searching matches text on a **prefix**, numbers and dates on **equality**, and — for `number`, `money` and `date` — on a **range** when you fill in both ends.

Field declarations

Key	Meaning
<code>type</code>	See below. Default <code>"string"</code>
<code>title</code>	The column heading and the input's label
<code>name</code>	The column in the table
<code>validate</code>	An array of validators, as in a <code>Form</code>
<code>inList</code>	Show this field in the list. Default <code>true</code>
<code>inSearch</code>	Offer this field in Search. Default <code>true</code>
<code>readOnly</code>	Show it, but do not let it be edited. Default <code>false</code>
<code>options</code>	For <code>list</code> : strings, or <code>{ label, value }</code> objects
<code>rows</code>	For <code>textarea</code> . Default 4
<code>onValue</code> / <code>offValue</code>	For <code>checkbox</code> : what to store. Default <code>1</code> and <code>0</code>

Types: `string`, `email`, `tel`, `number`, `money`, `date`, `textarea`, `checkbox`, `list`.

`date` uses the platform's own date picker — bigger touch targets, the right locale, and accessibility for free.

`money` renders with thousands separators and two decimals, and right-aligns with the other numbers.

Child tables

`linked` renders a child `TableMaint` inside each record, showing only the rows that belong to it and filling in the link column on save:

```
<${TableMaint} db=${db} title="Directory" table="person" fields=${PERSON_FIELDS}
  linked=${[
    { title: "Phone numbers", table: "person_tel", on: "person",
      fields: PHONE_FIELDS },
  ]} />
```

`on` is the child column holding the parent's row id. That column is hidden in the child's own views — which record you are looking at already says what it would hold.

Hooks

Each may be `async`, and each is optional.

Prop	When it runs
<code>preNew(values)</code>	Before an insert; may adjust <code>values</code> in place
<code>postNew(rowid, values)</code>	After an insert
<code>preEdit(id, values)</code>	Before an update
<code>postEdit(id, values)</code>	After an update
<code>preDelete(id)</code>	Before a delete
<code>postDelete(id)</code>	After a delete
<code>validateForm(values, id)</code>	Return <code>false</code> to refuse the save. <code>id</code> is null for a new record
<code>specialActions(id)</code>	Return <code>[{ label, bind }]</code> to add items to the record view's Special menu
<code>onError(e)</code>	Anything that went wrong; the banner shows it either way

Options from another table

```
const groups = useTableOptions(db, "grp", "name");
// ...
{ type: "list", title: "Group", name: "grp", options: groups }
```

It reads `rowid` and the named column, and gives you `{ label, value }` pairs with the row id as the value.

17. `acelery/sql.js` — SQLite

```
import { openDB, deleteDB } from "acelery/sql.js";

const db = await openDB("water.db");          // db/water.db, created if absent
await db.exec("create table if not exists drink (at text, ml integer)");

const id  = await db.insert("insert into drink values (?, ?)",
                           [new Date().toISOString(), 250]);
const rows = await db.select("select * from drink where ml > ?", [100]);
const one  = await db.selectOne("select sum(ml) as total from drink");
const n    = await db.exec("delete from drink where ml < ?", [50]);
```

Call	Resolves to
<code>openDB(path, basePath?)</code>	a <code>Database</code> . The file is created if it is not there
<code>deleteDB(path, basePath?)</code>	—
<code>db.select(sql, args?)</code>	every row, as an array of objects
<code>db.selectOne(sql, args?)</code>	the first row, or <code>null</code>
<code>db.exec(sql, args?)</code>	the number of rows changed
<code>db.insert(sql, args?)</code>	the new row id
<code>db.close()</code>	—

Everything is a promise, so the UI does not freeze while a statement runs. Columns keep their SQLite types: an INTEGER comes back a number, NULL comes back `null` . A failed statement rejects with SQLite's own message.

Rules worth following

- **Always pass values as ? parameters.** Never build SQL by joining strings: a name with an apostrophe in it will break the statement, and worse if the value came from outside your app.
- **Create your tables when the app starts**, with `create table if not exists` . The app has to work on a device where its database does not exist yet.
- **Open the database once** and pass it down. Opening it per screen means several handles contending for the same lock.

18. `acelery/file.js` — files

Paths are relative to aCelery's `files/` directory. Anything that resolves outside the aCelery folder is refused — a `..` that tries to climb out throws rather than reading somewhere it should not.

```
import * as file from "acelery/file.js";

const f = await file.open("Garden/notes.txt");
await f.write("planted the roses\n", true); // append
const text = await f.read();
await f.close();

await file.mkdir("Garden/photos");
const entries = await file.listFiles("Garden");
await file.writeBytes("Garden/rose.jpg", blob);
const src = file.url("Garden/rose.jpg"); // for <img src>
```

Call	Notes
<code>open(path, basePath?)</code>	A <code>FileHandle</code> , created if absent
<code>handle.read()</code>	The whole file, as text
<code>handle.write(text, append?)</code>	Truncates unless <code>append</code> is true
<code>handle.delete()</code>	
<code>handle.close()</code>	
<code>listFiles(path, basePath?)</code>	<code>[{ path, fname, directory, lastmodified, length }]</code>
<code>mkdir(path, basePath?)</code>	
<code>writeBytes(path, data, basePath?)</code>	A <code>Blob</code> , <code>File</code> , <code>ArrayBuffer</code> or <code>Uint8Array</code> . Folders are created as needed. Resolves to the size written
<code>url(path, basePath?)</code>	An address for <code></code> , a link or <code>fetch</code>
<code>externalStoragePath()</code>	The root <code>aCelery</code> will accept as a <code>basePath</code>

The text routes cannot carry binary data — use `writeBytes` for pictures and anything else that is not text. A single upload is capped at **25 MB**.

Keep an app's files in a folder named after it — `Garden/photos/12.jpg` — and store *that path* in the database, not the bytes.

19. `acelery/picker.js` — files and photos the user picks

Every picker here is the page's own file input, so a file comes from wherever the user is: the phone's gallery, camera or documents, and the other computer's disk when the app is open in a browser on the network. What you get back is a `File` either way.

```
import { pickFiles, pickImages, shrinkImage, cropImage } from "acelery/picker.js";
```

Call	Notes
<code>pickFiles({ accept?, multiple?, capture? })</code>	<code>File[]</code> , empty when the user cancels

Call	Notes
<code>pickImages({ multiple?, camera? })</code>	The photo picker, or with <code>camera: true</code> the camera
<code>shrinkImage(blob, { maxSide = 1600, type = "image/jpeg", quality = 0.85 })</code>	The whole picture, made small enough to store
<code>cropImage(blob, area, { maxSide = 1024, type, quality })</code>	Cuts <code>area</code> out and scales it. <code>area</code> is <code>{x, y, width, height}</code> in the picture's own pixels, as <code>ImageCropper</code> reports it, or <code>null</code> for the whole picture

Call the pickers from a tap. Browsers open a file picker only in answer to a click, never from `main` or an effect.

Prefer `FileButton` where the picker opens from a button. These functions click an input from script, and on iOS WebKit then puts its Photo Library / Take Photo / Choose File menu in the page's top-left corner rather than beside your button.

Shrink before you store. A photo from a phone camera is 3–12 MB; at the default `maxSide` it is a few hundred KB.

Choose the stored name yourself. The picked file's name comes from the user's device and may be anything at all.

```
const [photo] = await pickImages({ camera: true });
await file.writeBytes(`Garden/${Date.now()}.jpg`, await shrinkImage(photo));
```

20. `acelery/http.js` — the network

```
import * as http from "acelery/http.js";

const body = await http.get("https://example.com/rates.json");
const data = await http.getJson("https://example.com/rates.json");
const reply = await http.post("https://example.com/log", "a=1&b=2");
```

The device makes the request, not the page, so the page's origin does not restrict it and there is no CORS to satisfy. `post` sends a form-encoded body.

A failed request resolves to an empty string rather than rejecting — the behaviour aCelery has always had. Check for it: `getJson` on an empty body will throw a parse error, which is rarely the message you want to show.

21. `acelery/export.js` — sharing files and leaving the app

```
import { saveFile, exportProject, importProject, runApp, closeApp }
  from "acelery/export.js";

await saveFile("text/csv", "directory.csv", csv);
closeApp();
```

Call	What it does
<code>saveFile(mime, filename, text)</code>	Hands a file to the user: the share sheet on the device, a download in a browser
<code>exportProject(name)</code>	Zips one of your projects and offers it
<code>importProject()</code>	Opens the device's file picker to import a project zip
<code>runApp(title, app, debug?)</code>	Opens one of your apps
<code>closeApp()</code>	Leaves the running app and goes back to aCelery

These are the only calls that ask the device to *do* something rather than return data, so they are fire-and-forget — there is nothing to await except `saveFile`, which stages the file first.

In a browser on the network they still mean something: `saveFile` and `exportProject` become downloads, `runApp` opens the app in a new tab, and `closeApp` goes back. Only `importProject` has no remote form — it needs the device's own file picker — and it throws if you call it there.

22. `acelery/chart.js` — charts

A separate import, because Chart.js is 68 KB gzipped and most apps never draw one. An app pays for it only by asking.

```
import { Chart, fromRows } from "acelery/chart.js";

const rows = await db.select(
  "select grp, count(*) as people from person group by grp order by people desc");

<${Chart} type="bar" height=${260} data=${fromRows(rows, "grp", "people")} />
```

`fromRows(rows, labelColumn, valueColumns)` is the step between a result set and a Chart.js config — group in SQL, chart the rows. `valueColumns` may be an array for several series.

Chart props: `type` (`bar`, `line`, `pie` or `doughnut`), `data`, `options` (merged over the defaults), `height` (default 300), `title`, `className`.

Changing the data updates the chart in place, so a refresh animates from where it was rather than flashing.

23. `acelery/datatable.js` — sortable, searchable tables

[DataTables](#) 3, as a component. A separate import like the charts, because it is 55 KB gzipped and most apps never need it. The stylesheet comes with it: there is no `<link>` to add.

```
import { DataTable, DataTables, sqlSource } from "acelery/datatable.js";

// A table of any size: each page, sort and search is a query.
<${DataTable} source=${sqlSource(db, "person")}
  columns=${[
    { data: "mname", title: "Name" },
    { data: "email", title: "Email" },
    { data: "salary", title: "Salary", className: "text-end",
      render: DataTables.render.number(",", ".", 2) },
  ]}
  onClick=${(row) => openPerson(row.rowid)} />

// Rows you already have: sorted, searched and paged in the page.
<${DataTable} rows=${rows} columns=${[ { data: "name", title: "Name" } ]} />
```

`columns` uses DataTables' own vocabulary — `data`, `title`, `render`, `className`, `orderable`, `searchable`, `visible` — so its documentation applies as written.

`sqlSource(db, from)` answers DataTables from SQLite one page at a time, so a table of 50,000 rows never crosses the bridge in full. `from` is one of:

- a table name: `sqlSource(db, "person")`;
- `{ table, where, args }`: `sqlSource(db, { table: "person", where: "grp = ?", args: [g] })`;
- `{ query, where, args }`: any `select`, joins included. Its columns are what the table sorts and searches on. `args` bind the `?`s in `query`, then in `where`.

A table's rows carry their `rowid`, which is what `onClick` usually wants. For a view or a `WITHOUT ROWID` table, pass `rowid: false`. The search works the way DataTables' own does: every word has to appear in some column of the row, and "a quoted phrase" counts as one word. With a SQL source, only columns whose `data` is a plain column name can be sorted or searched. A computed column (`data: null` with a `render`) still shows, but its header doesn't offer to sort.

`DataTable` props: `columns`, `rows` or `source`, `onClick(row, event)` (Enter works too), `refresh` (change it to re-query the source, for example after an insert), `onError`, `onInit(api)` for the DataTables API, `options` (any DataTables option; read when the table is built), `striped`, `hover`, `small` and `className`.

Cells are text. A value containing `<b>` shows the characters `<b>`. A column that should render markup must supply its own `render`, and escaping whatever it inserts is then up to that function.

There is no jQuery. Most DataTables examples online write `$("#table").DataTable({...})`. That doesn't work here, because DataTables 3 doesn't need jQuery and aCelery doesn't ship it. Use the component, and pass the same options through `options`.

Errors, such as a misspelt table, show as a red alert above the table rather than a dialog. Narrow screens fold the columns that don't fit into a per-row details view instead of scrolling sideways.

24. Styling and theming your app

Your app renders inside the user's chosen theme, which may be any of the 18 and may be dark.

Style with Bootstrap 5's classes and CSS variables — `text-body-secondary`, `bg-body-tertiary`, `border`, `var(--bs-primary)` — and never with hard-coded colours. A hard-coded `#333` is invisible on Cyborg, and black text on a hard-coded white panel is the one thing that will look broken in dark mode.

- Every `.css` file in your project folder is loaded automatically.
- `ThemeSelect` gives the user a theme picker inside your app; `applyTheme(name)` sets one from code.
- `isDark()` tells you whether the current theme renders dark, and the document fires an `acelery:themechange` event when it changes.

Design for a phone first: one column, large touch targets, nothing that depends on hovering. `Col span` handles the wider screen.

Icons

aCelery ships a 34-glyph subset of Font Awesome 6, drawn with `<i class="fa-solid fa-play"></i>`. An icon outside the subset renders as nothing at all, so check before you rely on one. The subset is:

`arrow-left` · `arrow-up-right-from-square` · `circle-half-stroke` · `circle-info` · `clock-rotate-left` · `code` · `database` · `ellipsis-vertical` · `file` · `file-code` · `file-export` · `file-image` · `file-import` · `file-lines` · `floppy-disk` · `folder` · `gear` · `house` · `magnifying-glass` · `mobile-screen` · `moon` · `pen-to-square` · `play` · `plus` · `seedling` · `sun` · `table` · `table-cells` · `table-columns` · `terminal` · `trash` · `triangle-exclamation` · `wifi` · `xmark`

For anything else, use an emoji, an SVG of your own, or a picture in your project folder.

25. Debugging

On the device

If the module fails to parse, exports no entry function, or `main` throws, aCelery replaces the app with the error, its stack, and the file and line it happened on. Nothing is bundled or minified, so that is your line.

While the app is running, **Error log** in its menu shows everything it logged, every error nobody caught, and every promise nobody handled. It is in the menu on a *debug* run — which is what **Run** from Code does.

`console.log` works as it does anywhere. Prefer it to `alert`, which blocks the page.

From a browser on your computer

With **Share on this network** on, open `http://<phone>:8123/` on your computer and approve the pairing. You then have your app in a desktop browser, with DevTools: breakpoints, the element inspector, the network panel and a real console.

Your app is served at `/system/launcher.html?app=<App>`.

This is the fastest way to work on a screen's layout and logic. Come back to the device for anything that involves the camera, the share sheet or how it feels under a thumb.

The loop

1. Edit in Code, press **Run**.
2. Look at the screen; if it failed, read the stack.
3. **Reload** from the app's menu after each edit — it always loads the files as they are now.

Work in small steps: one change, one run, one look.

26. Limits, rules and gotchas

Names

- App names: letters, digits and underscore, 16 characters at most.
- Database names: letters, digits, dash and underscore.
- Table and column names in `TableMaint` must be ordinary SQL identifiers — they cannot be passed as parameters, so they are checked instead.

Sizes

- One `writeBytes` upload: 25 MB.
- The SQL scratchpad renders the first 200 rows, with **Show all** for the rest.

Things that catch people out

A picker that does nothing	You called it outside a click handler. Browsers open a picker only in answer to a tap
An icon that renders as nothing	It is outside the 34-glyph subset
<code>getJSON</code> throwing a parse error	The request failed; <code>http.get</code> resolves to <code>""</code> rather than rejecting
<code>TableMaint</code> erroring on load	The table does not exist yet. Create it before you render
A blank page, no error	The entry module exported the wrong thing. It must be the <i>default</i> export
Text invisible on some themes	A hard-coded colour. Use Bootstrap's classes and variables
Your app looks fine, the phone hangs	Something called <code>alert</code> , <code>confirm</code> or <code>prompt</code> in a loop. Use <code>Modal</code> instead
The assistant loses its connection (iOS)	<code>aCelery</code> left the screen. iOS suspends it; keep it open

Rules that are enforced, not advisory

- File and database paths cannot leave the `aCelery` folder.
- A device on the network sees nothing until you approve it.
- `importProject` needs the app; it has no form in a remote browser.

27. Troubleshooting

The app will not start, and the error names a line. Open that file in Code at that line. A stray `</>`, an unclosed template or a missing `${` before a component name is the usual cause.

Nothing renders and there is no error. Check that the entry module has `export default`, and that `render(..., document.body)` is actually called. A `main` that returns a component instead of rendering it does nothing.

The app runs but the data is empty. Confirm the table exists and holds rows: Data → your database → the table, or the SQL tab. If the table is missing, your `create table if not exists` is not running — usually because it is inside a branch that did not execute, or the app opened a different database file than you think.

Changes are not taking effect. Save the file (the dot beside its name means unsaved), then **Reload** the running app rather than leaving it on screen.

A computer cannot reach the phone. Both must be on the same Wi-Fi — a phone on mobile data is not reachable, and "guest" networks usually isolate devices from one another. Check the address in **Network access**; it changes when the phone rejoins a network.

The connection drops when you switch away. Expected on iPhone and iPad. On Android, sharing keeps working with the screen off.

You lost an assistant's key. Revoke it in **Approved devices** and create another; a key is shown only once.

You deleted something by mistake. There is no undo. Export projects you care about — the zip is the backup, and it imports back on any device.

Colophon

aCelery was written in 2014 by Xavier Llamas Rolland as an Android app, and rebuilt as a Flutter app for Android and iOS. It is free software under the [GNU General Public License v3.0](#).

The Example app on your device is the working reference for everything in Part 2 — open it in Code and read it top to bottom.

www.acelery.com